DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 2

UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS

COURSE: NATURAL LANGUAGE PROCESSING

NATURE: INDIVIDUAL ASSIGNMENT

TOTAL MARKS: 50

STUDENT NAME: N. Umasri

REGISTER NUMBER: 192424240:

CONTENTS AND SUBMISSION NOTE

This report answers the tasks in the corresponding uploaded assignment. It follows the task order and preserves the assignment's terminology. The worked examples explain the reasoning and the evidence expected.

Important: where the assignment requires a corpus experiment, annotation, code execution or human judgement, no unrun result is presented as an actual measurement. Such values must be collected by running the experiment and inserted into the report. Any sample calculation is explanatory only.

Q1 Tokenisation and Word Statistics

Concepts and direct answer

Tokenisation is a design decision, not a fixed fact. Define case folding, punctuation, contractions, hyphens, numbers/currency, emoticons and repeated-word rules before counting. Keep raw text and normalised tokens separately. Compare whitespace, punctuation-aware and refined policies on a cited corpus of at least 20,000 tokens. Record N, V, V/N, hapax count and top ten terms. Plot rank against frequency on log-log axes and estimate the Zipf slope. Tokenisation that splits contractions or hyphenated compounds changes vocabulary size and later n-gram sparsity. Use a clearly labelled ambiguous example such as 're-covered' versus 'recovered'.

The answer begins by identifying the input, the target interpretation and the assumptions. Terms are defined before they are used. This prevents the same label from being applied to different meanings in different parts of the analysis.

The main conclusion is that the system must make its intermediate decisions explicit. A final label or output alone is not enough to show that the method is correct.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q1 Tokenisation and Word Statistics

Representation and method

Tokenisation is a design decision, not a fixed fact. Define case folding, punctuation, contractions, hyphens, numbers/currency, emoticons and repeated-word rules before counting. Keep raw text and normalised tokens separately. Compare whitespace, punctuation-aware and refined policies on a cited corpus of at least 20,000 tokens. Record N, V, V/N, hapax count and top ten terms. Plot rank against frequency on log-log axes and estimate the Zipf slope. Tokenisation that splits contractions or hyphenated compounds changes vocabulary size and later n-gram sparsity. Use a clearly labelled ambiguous example such as 're-covered' versus 'recovered'.

A reliable design separates input preparation, representation, decision procedure and output. Each stage should have a stated data structure and a clear rule for updating it. Keep the original input so that normalisation does not erase information needed later.

For reproducibility, record the data source, assumptions, parameter values, software version and fixed random seed wherever applicable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q1 Tokenisation and Word Statistics

Worked example / trace

Tokenisation is a design decision, not a fixed fact. Define case folding, punctuation, contractions, hyphens, numbers/currency, emoticons and repeated-word rules before counting. Keep raw text and normalised tokens separately. Compare whitespace, punctuation-aware and refined policies on a cited corpus of at least 20,000 tokens. Record N, V, V/N, hapax count and top ten terms. Plot rank against frequency on log-log axes and estimate the Zipf slope. Tokenisation that splits contractions or hyphenated compounds changes vocabulary size and later n-gram sparsity. Use a clearly labelled ambiguous example such as 're-covered' versus 'recovered'.

Worked trace template: (1) identify the input units; (2) list candidate interpretations or structures; (3) apply the declared rule; (4) record the selected result and any unresolved alternative; (5) verify the result against a hand-worked example or gold annotation.

When two interpretations remain plausible, retain both or ask a clarification question. A forced choice without evidence can create a downstream error.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q1 Tokenisation and Word Statistics

Evaluation, limitations and conclusion

Tokenisation is a design decision, not a fixed fact. Define case folding, punctuation, contractions, hyphens, numbers/currency, emoticons and repeated-word rules before counting. Keep raw text and normalised tokens separately. Compare whitespace, punctuation-aware and refined policies on a cited corpus of at least 20,000 tokens. Record N, V, V/N, hapax count and top ten terms. Plot rank against frequency on log-log axes and estimate the Zipf slope. Tokenisation that splits contractions or hyphenated compounds changes vocabulary size and later n-gram sparsity. Use a clearly labelled ambiguous example such as 're-covered' versus 'recovered'.

Evaluation should include a normal case, a boundary case and an adversarial case. Report expected behaviour, actual behaviour, error category and a specific refinement. Use the same test inputs when comparing two designs.

Conclusion: select a design only when its measured behaviour meets the task requirements. Discuss remaining limitations and the conditions under which a simpler alternative may be preferable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q2 Unsmoothed N-grams

Concepts and direct answer

The chain rule is P(w1...wn)=product P(wi|w1...wi-1). Unigram models ignore history; bigrams condition on one previous token; trigrams condition on two. MLE: P(w)=C(w)/N; P(wi|wi-1)=C(wi-1,wi)/C(wi-1); trigram MLE divides trigram count by its two-word history count. Unseen n-grams receive zero probability. As n increases, distinct n-gram entries and sparsity generally increase, and test sentences with zero probability become more common. Compare in-domain tickets, out-of-domain news and an edge dataset using a fixed 80/20 split and seed.

The answer begins by identifying the input, the target interpretation and the assumptions. Terms are defined before they are used. This prevents the same label from being applied to different meanings in different parts of the analysis.

The main conclusion is that the system must make its intermediate decisions explicit. A final label or output alone is not enough to show that the method is correct.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q2 Unsmoothed N-grams

Representation and method

The chain rule is P(w1...wn)=product P(wi|w1...wi-1). Unigram models ignore history; bigrams condition on one previous token; trigrams condition on two. MLE: P(w)=C(w)/N; P(wi|wi-1)=C(wi-1,wi)/C(wi-1); trigram MLE divides trigram count by its two-word history count. Unseen n-grams receive zero probability. As n increases, distinct n-gram entries and sparsity generally increase, and test sentences with zero probability become more common. Compare in-domain tickets, out-of-domain news and an edge dataset using a fixed 80/20 split and seed.

A reliable design separates input preparation, representation, decision procedure and output. Each stage should have a stated data structure and a clear rule for updating it. Keep the original input so that normalisation does not erase information needed later.

For reproducibility, record the data source, assumptions, parameter values, software version and fixed random seed wherever applicable.

Evidence	**Required content**
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q2 Unsmoothed N-grams

Worked example / trace

The chain rule is P(w1...wn)=product P(wi|w1...wi-1). Unigram models ignore history; bigrams condition on one previous token; trigrams condition on two. MLE: P(w)=C(w)/N; P(wi|wi-1)=C(wi-1,wi)/C(wi-1); trigram MLE divides trigram count by its two-word history count. Unseen n-grams receive zero probability. As n increases, distinct n-gram entries and sparsity generally increase, and test sentences with zero probability become more common. Compare in-domain tickets, out-of-domain news and an edge dataset using a fixed 80/20 split and seed.

Worked trace template: (1) identify the input units; (2) list candidate interpretations or structures; (3) apply the declared rule; (4) record the selected result and any unresolved alternative; (5) verify the result against a hand-worked example or gold annotation.

When two interpretations remain plausible, retain both or ask a clarification question. A forced choice without evidence can create a downstream error.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q2 Unsmoothed N-grams

Evaluation, limitations and conclusion

The chain rule is P(w1...wn)=product P(wi|w1...wi-1). Unigram models ignore history; bigrams condition on one previous token; trigrams condition on two. MLE: P(w)=C(w)/N; P(wi|wi-1)=C(wi-1,wi)/C(wi-1); trigram MLE divides trigram count by its two-word history count. Unseen n-grams receive zero probability. As n increases, distinct n-gram entries and sparsity generally increase, and test sentences with zero probability become more common. Compare in-domain tickets, out-of-domain news and an edge dataset using a fixed 80/20 split and seed.

Evaluation should include a normal case, a boundary case and an adversarial case. Report expected behaviour, actual behaviour, error category and a specific refinement. Use the same test inputs when comparing two designs.

Conclusion: select a design only when its measured behaviour meets the task requirements. Discuss remaining limitations and the conditions under which a simpler alternative may be preferable.

Evidence	**Required content**
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q3 Smoothing

Concepts and direct answer

Add-one probability is (C(h,w)+1)/(C(h)+V); add-k uses a smaller positive pseudo-count. Backoff discounts observed higher-order counts and assigns reserved mass to a lower-order distribution. Deleted interpolation combines unigram, bigram and trigram estimates with non-negative weights summing to one. Estimate weights on held-out data. Check that conditional probabilities sum to one for at least ten histories. Record perplexity, unseen-bigram mass, learned lambda weights and adjusted counts. Add-one often performs poorly because it spreads too much probability uniformly across a large vocabulary.

The answer begins by identifying the input, the target interpretation and the assumptions. Terms are defined before they are used. This prevents the same label from being applied to different meanings in different parts of the analysis.

The main conclusion is that the system must make its intermediate decisions explicit. A final label or output alone is not enough to show that the method is correct.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q3 Smoothing

Representation and method

Add-one probability is (C(h,w)+1)/(C(h)+V); add-k uses a smaller positive pseudo-count. Backoff discounts observed higher-order counts and assigns reserved mass to a lower-order distribution. Deleted interpolation combines unigram, bigram and trigram estimates with non-negative weights summing to one. Estimate weights on held-out data. Check that conditional probabilities sum to one for at least ten histories. Record perplexity, unseen-bigram mass, learned lambda weights and adjusted counts. Add-one often performs poorly because it spreads too much probability uniformly across a large vocabulary.

A reliable design separates input preparation, representation, decision procedure and output. Each stage should have a stated data structure and a clear rule for updating it. Keep the original input so that normalisation does not erase information needed later.

For reproducibility, record the data source, assumptions, parameter values, software version and fixed random seed wherever applicable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q3 Smoothing

Worked example / trace

Add-one probability is (C(h,w)+1)/(C(h)+V); add-k uses a smaller positive pseudo-count. Backoff discounts observed higher-order counts and assigns reserved mass to a lower-order distribution. Deleted interpolation combines unigram, bigram and trigram estimates with non-negative weights summing to one. Estimate weights on held-out data. Check that conditional probabilities sum to one for at least ten histories. Record perplexity, unseen-bigram mass, learned lambda weights and adjusted counts. Add-one often performs poorly because it spreads too much probability uniformly across a large vocabulary.

Worked trace template: (1) identify the input units; (2) list candidate interpretations or structures; (3) apply the declared rule; (4) record the selected result and any unresolved alternative; (5) verify the result against a hand-worked example or gold annotation.

When two interpretations remain plausible, retain both or ask a clarification question. A forced choice without evidence can create a downstream error.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q3 Smoothing

Evaluation, limitations and conclusion

Add-one probability is (C(h,w)+1)/(C(h)+V); add-k uses a smaller positive pseudo-count. Backoff discounts observed higher-order counts and assigns reserved mass to a lower-order distribution. Deleted interpolation combines unigram, bigram and trigram estimates with non-negative weights summing to one. Estimate weights on held-out data. Check that conditional probabilities sum to one for at least ten histories. Record perplexity, unseen-bigram mass, learned lambda weights and adjusted counts. Add-one often performs poorly because it spreads too much probability uniformly across a large vocabulary.

Evaluation should include a normal case, a boundary case and an adversarial case. Report expected behaviour, actual behaviour, error category and a specific refinement. Use the same test inputs when comparing two designs.

Conclusion: select a design only when its measured behaviour meets the task requirements. Discuss remaining limitations and the conditions under which a simpler alternative may be preferable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q4 Entropy and Perplexity

Concepts and direct answer

Entropy is H=-sum p(x)log2 p(x). Per-token cross-entropy is the negative average log probability assigned to test tokens. Perplexity is 2^H and can be interpreted as a weighted average branching factor. A uniform distribution over ten symbols has entropy log2(10) and perplexity 10; a skewed distribution has lower entropy. Compare unigram, bigram and trigram models on in-domain, out-of-domain and shuffled/repetitive text. Vocabulary and unknown-word policies must match before perplexities can be compared.

The answer begins by identifying the input, the target interpretation and the assumptions. Terms are defined before they are used. This prevents the same label from being applied to different meanings in different parts of the analysis.

The main conclusion is that the system must make its intermediate decisions explicit. A final label or output alone is not enough to show that the method is correct.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q4 Entropy and Perplexity

Representation and method

Entropy is H=-sum p(x)log2 p(x). Per-token cross-entropy is the negative average log probability assigned to test tokens. Perplexity is 2^H and can be interpreted as a weighted average branching factor. A uniform distribution over ten symbols has entropy log2(10) and perplexity 10; a skewed distribution has lower entropy. Compare unigram, bigram and trigram models on in-domain, out-of-domain and shuffled/repetitive text. Vocabulary and unknown-word policies must match before perplexities can be compared.

A reliable design separates input preparation, representation, decision procedure and output. Each stage should have a stated data structure and a clear rule for updating it. Keep the original input so that normalisation does not erase information needed later.

For reproducibility, record the data source, assumptions, parameter values, software version and fixed random seed wherever applicable.

Evidence	**Required content**
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q4 Entropy and Perplexity

Worked example / trace

Entropy is H=-sum p(x)log2 p(x). Per-token cross-entropy is the negative average log probability assigned to test tokens. Perplexity is 2^H and can be interpreted as a weighted average branching factor. A uniform distribution over ten symbols has entropy log2(10) and perplexity 10; a skewed distribution has lower entropy. Compare unigram, bigram and trigram models on in-domain, out-of-domain and shuffled/repetitive text. Vocabulary and unknown-word policies must match before perplexities can be compared.

Worked trace template: (1) identify the input units; (2) list candidate interpretations or structures; (3) apply the declared rule; (4) record the selected result and any unresolved alternative; (5) verify the result against a hand-worked example or gold annotation.

When two interpretations remain plausible, retain both or ask a clarification question. A forced choice without evidence can create a downstream error.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q4 Entropy and Perplexity

Evaluation, limitations and conclusion

Entropy is H=-sum p(x)log2 p(x). Per-token cross-entropy is the negative average log probability assigned to test tokens. Perplexity is 2^H and can be interpreted as a weighted average branching factor. A uniform distribution over ten symbols has entropy log2(10) and perplexity 10; a skewed distribution has lower entropy. Compare unigram, bigram and trigram models on in-domain, out-of-domain and shuffled/repetitive text. Vocabulary and unknown-word policies must match before perplexities can be compared.

Evaluation should include a normal case, a boundary case and an adversarial case. Report expected behaviour, actual behaviour, error category and a specific refinement. Use the same test inputs when comparing two designs.

Conclusion: select a design only when its measured behaviour meets the task requirements. Discuss remaining limitations and the conditions under which a simpler alternative may be preferable.

Evidence	**Required content**
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q5 Word Classes and Tagsets

Concepts and direct answer

Open classes include nouns, lexical verbs, adjectives and many adverbs; closed classes include determiners, pronouns, prepositions, conjunctions and auxiliaries. Compare a coarse Universal POS-style tagset, Penn Treebank and a custom support-chat tagset that can identify product names, abbreviations, emoticons and code-switching. Define annotation policy and double-annotate at least 100 sentences. Cohen's kappa is (observed agreement - chance agreement)/(1 - chance agreement). Report ambiguity, average tags per type and common confusions.

The answer begins by identifying the input, the target interpretation and the assumptions. Terms are defined before they are used. This prevents the same label from being applied to different meanings in different parts of the analysis.

The main conclusion is that the system must make its intermediate decisions explicit. A final label or output alone is not enough to show that the method is correct.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q5 Word Classes and Tagsets

Representation and method

Open classes include nouns, lexical verbs, adjectives and many adverbs; closed classes include determiners, pronouns, prepositions, conjunctions and auxiliaries. Compare a coarse Universal POS-style tagset, Penn Treebank and a custom support-chat tagset that can identify product names, abbreviations, emoticons and code-switching. Define annotation policy and double-annotate at least 100 sentences. Cohen's kappa is (observed agreement - chance agreement)/(1 - chance agreement). Report ambiguity, average tags per type and common confusions.

A reliable design separates input preparation, representation, decision procedure and output. Each stage should have a stated data structure and a clear rule for updating it. Keep the original input so that normalisation does not erase information needed later.

For reproducibility, record the data source, assumptions, parameter values, software version and fixed random seed wherever applicable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q5 Word Classes and Tagsets

Worked example / trace

Open classes include nouns, lexical verbs, adjectives and many adverbs; closed classes include determiners, pronouns, prepositions, conjunctions and auxiliaries. Compare a coarse Universal POS-style tagset, Penn Treebank and a custom support-chat tagset that can identify product names, abbreviations, emoticons and code-switching. Define annotation policy and double-annotate at least 100 sentences. Cohen's kappa is (observed agreement - chance agreement)/(1 - chance agreement). Report ambiguity, average tags per type and common confusions.

Worked trace template: (1) identify the input units; (2) list candidate interpretations or structures; (3) apply the declared rule; (4) record the selected result and any unresolved alternative; (5) verify the result against a hand-worked example or gold annotation.

When two interpretations remain plausible, retain both or ask a clarification question. A forced choice without evidence can create a downstream error.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q5 Word Classes and Tagsets

Evaluation, limitations and conclusion

Open classes include nouns, lexical verbs, adjectives and many adverbs; closed classes include determiners, pronouns, prepositions, conjunctions and auxiliaries. Compare a coarse Universal POS-style tagset, Penn Treebank and a custom support-chat tagset that can identify product names, abbreviations, emoticons and code-switching. Define annotation policy and double-annotate at least 100 sentences. Cohen's kappa is (observed agreement - chance agreement)/(1 - chance agreement). Report ambiguity, average tags per type and common confusions.

Evaluation should include a normal case, a boundary case and an adversarial case. Report expected behaviour, actual behaviour, error category and a specific refinement. Use the same test inputs when comparing two designs.

Conclusion: select a design only when its measured behaviour meets the task requirements. Discuss remaining limitations and the conditions under which a simpler alternative may be preferable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q6 POS Tagging Approaches

Concepts and direct answer

A most-frequent-tag baseline assigns the most common training tag to each known word. A rule-based tagger uses a lexicon and at least ten contextual constraints. A bigram HMM uses transition and emission probabilities and Viterbi decoding; unknown words can use suffix, capitalization and numeric classes. A Brill tagger learns ordered correction rules from templates. Compare overall, known-word and unknown-word accuracy, confusion matrices, transformation counts and Viterbi transitions at 10%, 50% and 100% training data.

The answer begins by identifying the input, the target interpretation and the assumptions. Terms are defined before they are used. This prevents the same label from being applied to different meanings in different parts of the analysis.

The main conclusion is that the system must make its intermediate decisions explicit. A final label or output alone is not enough to show that the method is correct.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q6 POS Tagging Approaches

Representation and method

A most-frequent-tag baseline assigns the most common training tag to each known word. A rule-based tagger uses a lexicon and at least ten contextual constraints. A bigram HMM uses transition and emission probabilities and Viterbi decoding; unknown words can use suffix, capitalization and numeric classes. A Brill tagger learns ordered correction rules from templates. Compare overall, known-word and unknown-word accuracy, confusion matrices, transformation counts and Viterbi transitions at 10%, 50% and 100% training data.

A reliable design separates input preparation, representation, decision procedure and output. Each stage should have a stated data structure and a clear rule for updating it. Keep the original input so that normalisation does not erase information needed later.

For reproducibility, record the data source, assumptions, parameter values, software version and fixed random seed wherever applicable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q6 POS Tagging Approaches

Worked example / trace

A most-frequent-tag baseline assigns the most common training tag to each known word. A rule-based tagger uses a lexicon and at least ten contextual constraints. A bigram HMM uses transition and emission probabilities and Viterbi decoding; unknown words can use suffix, capitalization and numeric classes. A Brill tagger learns ordered correction rules from templates. Compare overall, known-word and unknown-word accuracy, confusion matrices, transformation counts and Viterbi transitions at 10%, 50% and 100% training data.

Worked trace template: (1) identify the input units; (2) list candidate interpretations or structures; (3) apply the declared rule; (4) record the selected result and any unresolved alternative; (5) verify the result against a hand-worked example or gold annotation.

When two interpretations remain plausible, retain both or ask a clarification question. A forced choice without evidence can create a downstream error.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q6 POS Tagging Approaches

Evaluation, limitations and conclusion

A most-frequent-tag baseline assigns the most common training tag to each known word. A rule-based tagger uses a lexicon and at least ten contextual constraints. A bigram HMM uses transition and emission probabilities and Viterbi decoding; unknown words can use suffix, capitalization and numeric classes. A Brill tagger learns ordered correction rules from templates. Compare overall, known-word and unknown-word accuracy, confusion matrices, transformation counts and Viterbi transitions at 10%, 50% and 100% training data.

Evaluation should include a normal case, a boundary case and an adversarial case. Report expected behaviour, actual behaviour, error category and a specific refinement. Use the same test inputs when comparing two designs.

Conclusion: select a design only when its measured behaviour meets the task requirements. Discuss remaining limitations and the conditions under which a simpler alternative may be preferable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q7 Diagnose, Redesign and Validate

Concepts and direct answer

Choose a measured bottleneck such as n-gram table growth, Viterbi transitions, add-one probability waste or unknown-word errors. Diagnose where memory or computation is spent, define a measurable objective, redesign without simply replacing the method with a neural model, give pseudocode, predict improvement, then compare original and revised systems under identical conditions. Report counted quantities such as table entries, transitions, rules, perplexity and accuracy. Label unrun values as illustrative.

The answer begins by identifying the input, the target interpretation and the assumptions. Terms are defined before they are used. This prevents the same label from being applied to different meanings in different parts of the analysis.

The main conclusion is that the system must make its intermediate decisions explicit. A final label or output alone is not enough to show that the method is correct.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q7 Diagnose, Redesign and Validate

Representation and method

Choose a measured bottleneck such as n-gram table growth, Viterbi transitions, add-one probability waste or unknown-word errors. Diagnose where memory or computation is spent, define a measurable objective, redesign without simply replacing the method with a neural model, give pseudocode, predict improvement, then compare original and revised systems under identical conditions. Report counted quantities such as table entries, transitions, rules, perplexity and accuracy. Label unrun values as illustrative.

A reliable design separates input preparation, representation, decision procedure and output. Each stage should have a stated data structure and a clear rule for updating it. Keep the original input so that normalisation does not erase information needed later.

For reproducibility, record the data source, assumptions, parameter values, software version and fixed random seed wherever applicable.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q7 Diagnose, Redesign and Validate

Worked example / trace

Choose a measured bottleneck such as n-gram table growth, Viterbi transitions, add-one probability waste or unknown-word errors. Diagnose where memory or computation is spent, define a measurable objective, redesign without simply replacing the method with a neural model, give pseudocode, predict improvement, then compare original and revised systems under identical conditions. Report counted quantities such as table entries, transitions, rules, perplexity and accuracy. Label unrun values as illustrative.

Worked trace template: (1) identify the input units; (2) list candidate interpretations or structures; (3) apply the declared rule; (4) record the selected result and any unresolved alternative; (5) verify the result against a hand-worked example or gold annotation.

When two interpretations remain plausible, retain both or ask a clarification question. A forced choice without evidence can create a downstream error.

Evidence	Required content
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Q7 Diagnose, Redesign and Validate

Evaluation, limitations and conclusion

Choose a measured bottleneck such as n-gram table growth, Viterbi transitions, add-one probability waste or unknown-word errors. Diagnose where memory or computation is spent, define a measurable objective, redesign without simply replacing the method with a neural model, give pseudocode, predict improvement, then compare original and revised systems under identical conditions. Report counted quantities such as table entries, transitions, rules, perplexity and accuracy. Label unrun values as illustrative.

Evaluation should include a normal case, a boundary case and an adversarial case. Report expected behaviour, actual behaviour, error category and a specific refinement. Use the same test inputs when comparing two designs.

Conclusion: select a design only when its measured behaviour meets the task requirements. Discuss remaining limitations and the conditions under which a simpler alternative may be preferable.

Evidence	**Required content**
Input	Sentence, corpus, query or scenario
Method	Formula, grammar, rule or algorithm
Trace	Intermediate steps and selected output
Validation	Metric, counterexample or comparison

Additional Analysis – Engineering trade-offs

Discuss accuracy, memory, latency, transparency, annotation cost and maintainability. A method is useful only relative to its operating constraints.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Final recommendation

Choose the design that best satisfies the assignment's requirements and justify it using traceable evidence rather than a general claim that one method is better.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Data and assumptions

State the data domain, language variety, preprocessing, annotation rules and excluded cases. Assumptions must be visible because they affect the interpretation of every result.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Reproducibility

Record corpus source, software and library versions, random seed, parameters and train/test split. Save intermediate data and final output tables.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Baseline comparison

A simple baseline provides a reference point. Compare every improvement with the same baseline, data and evaluation metric.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Error analysis

Group errors by their cause, such as ambiguity, missing vocabulary, incorrect structure, weak context, annotation disagreement or domain shift.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Boundary tests

Test empty input, one-token input, long input, unknown terms, punctuation variation and conflicting context. These cases expose hidden assumptions.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Adversarial tests

Construct a difficult input that targets the method's weakest assumption. Predict the failure before testing and compare prediction with observation.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Metric interpretation

Define every metric and denominator. Aggregate accuracy can hide poor performance on rare categories, unknown words or out-of-domain text.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Limitations

Separate data limitations from algorithm limitations. Explain which problem can be fixed with better examples and which requires a design change.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Engineering trade-offs

Discuss accuracy, memory, latency, transparency, annotation cost and maintainability. A method is useful only relative to its operating constraints.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Final recommendation

Choose the design that best satisfies the assignment's requirements and justify it using traceable evidence rather than a general claim that one method is better.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Data and assumptions

State the data domain, language variety, preprocessing, annotation rules and excluded cases. Assumptions must be visible because they affect the interpretation of every result.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Reproducibility

Record corpus source, software and library versions, random seed, parameters and train/test split. Save intermediate data and final output tables.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Baseline comparison

A simple baseline provides a reference point. Compare every improvement with the same baseline, data and evaluation metric.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Error analysis

Group errors by their cause, such as ambiguity, missing vocabulary, incorrect structure, weak context, annotation disagreement or domain shift.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Boundary tests

Test empty input, one-token input, long input, unknown terms, punctuation variation and conflicting context. These cases expose hidden assumptions.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Adversarial tests

Construct a difficult input that targets the method's weakest assumption. Predict the failure before testing and compare prediction with observation.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Metric interpretation

Define every metric and denominator. Aggregate accuracy can hide poor performance on rare categories, unknown words or out-of-domain text.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Limitations

Separate data limitations from algorithm limitations. Explain which problem can be fixed with better examples and which requires a design change.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

Additional Analysis – Engineering trade-offs

Discuss accuracy, memory, latency, transparency, annotation cost and maintainability. A method is useful only relative to its operating constraints.

Application to this assignment: UNIT II: WORD LEVEL AND SYNTACTIC ANALYSIS. The design should preserve the input, expose intermediate decisions, state uncertainty and allow each stage to be checked independently.

Suggested evidence record: Input | Expected result | Actual result | Error category | Proposed correction. Complete this record using the student's own execution or annotation; do not present an illustrative value as a measured result.

CONCLUSION

The answers in this report follow the structure and concepts of the uploaded assignment. The central engineering lesson is to define representations and decision rules, trace the reasoning, test difficult cases, measure performance and defend the final design with evidence.

REFERENCES AND ACKNOWLEDGEMENT

Primary source: the corresponding DSA03 Natural Language Processing assignment PDF supplied for this task. Add exact textbooks, papers, corpora, toolkits and software references actually consulted, including author or organisation, year, title and publication details.

Disclose AI or reference tools used as required by the assignment. Verify the examples and replace all illustrative experimental values with results from your own implementation, corpus analysis or annotation.